

Relatório de Integração – Uso de MOM com RabbitMQ

Para a implementação do sistema de agendamento de viagens em grupo, foi adotado um middleware orientado a mensagens (MOM – *Message Oriented Middleware*) com o objetivo de desacoplar os processos de comunicação entre os módulos da aplicação. O sistema possui um fluxo fortemente baseado em eventos, principalmente devido às negociações entre clientes e motoristas, que envolvem propostas, contrapropostas, aceitações e notificações assíncronas.

A ferramenta escolhida foi o RabbitMQ, devido à sua ampla adoção no mercado, facilidade de integração com aplicações back-end e suporte ao protocolo AMQP (*Advanced Message Queuing Protocol*). Outro fator relevante para a escolha foi a possibilidade de implementar filas, exchanges, roteamento de mensagens e mecanismos de tolerância a falhas, permitindo maior escalabilidade e confiabilidade ao sistema.

O padrão arquitetural utilizado foi o modelo orientado a eventos (*Event-Driven Architecture*), no qual os serviços publicam eventos relacionados às ações do sistema e outros componentes realizam o consumo dessas mensagens de forma assíncrona. Dessa forma, ao ocorrer uma ação como a criação de uma solicitação de viagem ou uma contraproposta de um motorista, um evento é publicado no RabbitMQ e distribuído aos consumidores responsáveis pelo processamento correspondente, como envio de notificações, atualização de histórico e gerenciamento das negociações.

Para o roteamento das mensagens, foi considerada a utilização de exchanges do tipo *topic*, permitindo maior flexibilidade na organização dos eventos do sistema. Esse modelo facilita a expansão futura da aplicação, pois novos consumidores podem ser adicionados sem necessidade de alteração direta na lógica principal da API.

Entre os principais desafios encontrados durante o desenvolvimento, destaca-se a definição adequada do fluxo de estados da negociação entre cliente e motorista. Como o processo depende da confirmação de ambas as partes, tornou-se necessário estabelecer regras claras de sincronização e consistência das informações.

Além disso, também foi considerada a necessidade de mecanismos de resiliência, como filas de erro (*Dead Letter Queue*) e políticas de repetição (*retry*), visando evitar perda de mensagens em casos de falha temporária dos consumidores. Essas estratégias contribuem para aumentar a confiabilidade da comunicação assíncrona entre os módulos do sistema.

Por fim, a utilização do MOM permitiu uma arquitetura mais desacoplada, escalável e preparada para futuras evoluções, tornando o sistema mais adequado para cenários com grande volume de eventos e múltiplos serviços consumidores.